Sana'a University
Faculty of Computer and Information Technology
Master of Cyber Security

جامعة صنعاء
كلية الحاسوب وتكنلوجيا المعلومات
ماجستير الأمن السيبراني





Lightweight Behavior-Based Ransomware Detection and Automated Response Using Native Windows PowerShell in Resource-Constrained Environments

Prepared by
Rafah Hameed Saleh Hassan

Supervisor
Dr. Ahmed Shalabi

First Semester
June 2026 24

Research Proposal for the Degree of Research Methodology Course

Abstract

Ransomware remains a widespread cybersecurity issue because of the ability to quickly encrypt the files, to damage data integrity and disrupts availability of the system. Conventional methods of ransomware detection may be ineffective when it comes to emergent ransomware varieties, whereas some more modern detection solutions make use of time and resources-consuming approaches or additional software components. In this paper, this research presents a lightweight approach to ransomware detection and automation based only on native tools of Windows PowerShell.

The presented approach to ransomware monitoring makes use of event-driven architecture, which helps detect suspicious behavior through real-time analysis of file-system operations according to two main criteria of detecting ransomware – quick modifications of the files and changes in their extension. This process is implemented through PowerShell's FileSystemWatcher, which observes creation, modification, and renaming of the files in a specific directory. Once predefined behavioral criteria indicate suspicious activity, automated containment is implemented by stopping all PowerShell processes except for the monitoring ones. Also, the system records security events in the CSV-log, provides local Windows notifications and sends notifications to the administrator through e-mail. Such a system was implemented experimentally within a controlled Windows 10 environment through using 100 test files and PowerShell-based ransomware simulation script. Testing included unprotected ransomware activity, attack with protection, ransomware content-only modifications without changing file extensions and benign operations with files performed quickly.

Test results show that unprotected activity leads to encrypting all 100 test files within 22.78 seconds. With the usage of the presented solution, detection and containment take place in 1.26 seconds on average and restrict the number of impacted files to one and keep 99% of test files from getting encrypted. In the case of content-only modifications of the files, suspicious actions were detected in 3 seconds and left 90% of the files safe. In a test case for benign activity, only one alert took place. The average delivery time for notifications via email is 4.73 seconds. At the same time, during idle monitoring mode, CPU utilization is under 1%, and in the active mode of detection and containment, the figure is 3.5%.

Contents

Abstract **i**

List of Tables **iv**

Chapter 1: Introduction **1**

1.1 Introduction 2

1.2 Problem Statement 2

1.3 Purpose Statement 3

1.4 Research Questions 3

1.5 Objectives 4

1.6 Significance of the Study 4

1.7 Scope and Delimitations 4

1.8 Limitations 5

1.9 Definition of Key Terms 5

1.10 Organization of the Thesis 6

Chapter 2: Literature Review **7**

2.1 Introduction 8

2.2 Overview of Ransomware Threats 8

2.3 Ransomware Detection Techniques 9

2.3.1 Signature-Based Detection 9

2.3.2 Machine Learning and AI Approaches 10

2.3.3 Behavior-Based Detection 10

2.4 Endpoint Security and Native OS Tools 10

2.5 Comparative Analysis of Related Studies 11

2.6 Research Gap 11

Chapter 3: Methodology **13**

3.1 Introduction 14

3.2 Nature of the Quantitative Research 14

3.3 Quantitative Research Design 15

3.4 Study Population, Experimental Units, and Test Data 17

3.5 Measurement Instruments and Operational Definitions 17

3.6 Data Collection Procedures 19

3.7 Experimental Environment and Implementation Details 21

3.8 Experimental Design and Protocol 23

3.9 Baseline Selection and Comparison Strategy 25

3.10 Statistical and Experimental Analysis Plan 26

3.11 Ethical and Safety Considerations 27

3.12 Limitations and Threats to Validity 28

3.13 Chapter Summary 29

List of Tables

1 Comparative Analysis of Recent Studies 11

2 Quantitative Research Design 16

3 Experimental Units and Data Sources 18

4 Operational Definitions and Measurement Methods 20

5 Data Collection Procedure 21

6 Experimental Environment and Implementation Configuration 22

7 Experimental Scenarios 24

8 Baseline and Experimental Comparison 25

9 Research Questions and Analysis Plan 28

Chapter 1: Introduction

1.1 Introduction

Nowadays, the protection of digital information becomes vital for all people and companies. One of the rising threats of the modern world is ransomware – a type of malware which quickly encrypts user or corporate files making them inaccessible. These quick attacks cause disruption of the routine workflow of companies and result in considerable losses for the business.

However, today some systems have already been developed in order to protect computers and companies from these kinds of attacks. Some systems are based on identification of signatures of the malware while other use more advanced computational tools, including machine learning. Although those can be quite effective, all such systems require considerable computer resources and power supply, what makes them ineffective in resource poor settings where people do not have an opportunity to use expensive computing equipment. Moreover, some alternative solutions which detect the suspicious file activities may produce a lot of false alarms or require additional software to be installed.

Therefore, there is a need in a simple and lightweight system. The purpose of the project is to secure computers without slowing down their work.

The purpose of this project is to develop a lightweight and fast system which automatically detects ransomware attacks and mitigates them. This system will be created with the help of native Windows PowerShell utility only. The system will constantly observe file actions and as soon as it finds out suspicious rapid file actions or appearance of strange file extensions, it will identify the possible ransomware attacks.

If an attack is detected, this system will quickly stop malicious processes, register the attack and notify system administrators via emails and screens. It should be noted that one of the important benefits of the proposed system is its independency from any additional software.

1.2 Problem Statement

This research tackles the issue of ransomware protection without impacting system performance. Nonetheless, the current ransomware protection approaches cannot claim their optimality in all cases.

The traditional ransomware protection strategies based on signature analysis are not able to recognize new ransomware threats because they cannot be previously recognized as such. On the other hand, more advanced ransomware protection techniques with the usage of machine learning algorithms can recognize new ransomware

variants but they demand a lot of processing power making them unusable in most situations.

Ransomware detection based on behavior analysis that observes system activities during execution presents a perspective approach to this problem; however, it faces many problems in practice. Most approaches have a high rate of false positives or force users to use some third-party programs.

In conclusion, there is an evident tradeoff between ransomware detection accuracy and system performance. The main contribution of this research is a development of the lightweight ransomware detection solution running completely using the built-in Windows tools only (PowerShell).

1.3 Purpose Statement

The purpose of this experimental study is to develop and evaluate a lightweight, behavior-based ransomware detection and automated response system using native Windows PowerShell.

This study will monitor file activities among local computer files in a controlled Windows 10 environment to protect resource-constrained systems without using third-party software.

1.4 Research Questions

- How effectively can a native Windows PowerShell system detect and stop ransomware in resource-constrained environments?

1. How accurately does the proposed system identify ransomware behavior by monitoring rapid file changes and unusual file extensions?
2. How fast does the automated response mechanism contain the attack to protect user data from being encrypted?
3. What is the impact of this monitoring system on computer resources (like CPU usage) and false positive rates during normal daily tasks?

1.5 Objectives

- To develop and evaluate a lightweight, behavior-based ransomware detection and automated response system using native Windows PowerShell in resource-constrained environments.

1. To determine how accurately the system identifies ransomware behavior by monitoring rapid file changes and unusual file extensions.
2. To measure how fast the automated response mechanism stops the attack and protects user files.
3. To assess the impact of the monitoring system on computer resources (like CPU usage) and calculate the false positive rates during normal daily tasks

1.6 Significance of the Study

This study is significant at several levels.At the theoretical level,it adds to the existing knowledge of endpoint security. It proves that native operating system tools can effectively detect ransomware behavior without relying on heavy machine learning models or complex external libraries.At the practical level, the findings directly benefit IT teams, system administrators, and organizations with limited budgets. By using this proposed system, they can protect their computers, contain attacks in real-time, and prevent data loss without buying or installing expensive third-party security software. At the methodological level, the study introduces a clean, event-driven framework using Windows PowerShell. Other researchers and software engineers can easily adopt, modify, or expand this framework to build similar lightweight security tools in the future.At the contextual level, this study is extremely valuable for institutions and developing regions that suffer from limited hardware resources. It provides a highly efficient, low-cost defense strategy that can be quickly deployed in real-world environments where computer processing power and financial capabilities are constrained.

1.7 Scope and Delimitations

This study is delimited to local file system activities on standalone Windows 10 workstations within a controlled environment at resource-constrained environments. It focuses on detecting ransomware behavior specifically rapid file modifications and abnormal extension changes using native Windows PowerShell, and does not cover

network based attacks or other operating systems like macOS and Linux. The study does not include third-party security software or heavy machine learning models, as the major goal of this study is to obtain an in depth understanding of lightweight ransomware detection in resource constrained environments.

1.8 Limitations

This study has several limitations that should be considered when interpreting the findings. First, the evaluation was performed in a controlled, isolated laboratory environment using a specific dataset of 100 test files under simulated attack scenarios. This might not fully represent the complex, dynamic activities and data high-loads found in real-world enterprise environments. Second, because the system relies on native Windows PowerShell features like *FileSystemWatcher*, its operation is entirely dependent on the stability and response time of the underlying Windows operating system APIs. Therefore, extreme system delays or highly sophisticated ransomware that works at the kernel level might affect the overall speed of the automated response. As a result, these factors may limit the generalizability of the performance metrics across large-scale corporate networks. Nevertheless, these limitations also point to opportunities for future research, such as expanding the framework to support multi-directory monitoring, testing the system against more diverse ransomware variants under high-load settings, and integrating lightweight machine learning models to improve detection accuracy.

1.9 Definition of Key Terms

The following terms are defined to clarify how they are used and measured within the specific context of this study:

- **Automated Response** As an idea, it implies a reaction to a threat carried out without human intervention. In the context of this research, it is understood as the actions triggered immediately after the detection of ransomware by the script including terminating suspicious processes, logging, and sending emails.
- **Behavior-Based Detection** It is a technique of identifying potential security threats through the observation of the way of acting of suspicious programs, not by searching for the signature of a virus. In this study, it is operationalized as the continuous monitoring of file system for abnormal speeds of file modification and unexpected file extension changes.

- **False Positive** It is a case when a non-threatening behavior is misidentified as cyberattack. In the context of this study, it implies the alerting or response generated during normal file operations performed by a user at high speed.
- **File Extension Change** It implies changing the file format (for instance, renaming .txt to .locked). In this proposed system, unexpected changes of file extension serve as a direct indicator of potential ransomware activity.
- **Native Windows PowerShell** A command line shell and scripting tool provided as part of Microsoft Windows operating systems. In this study, it implies the single tool utilized for implementing the whole solution, without resorting to third-party software.
- **Ransomware** Malicious software that encrypts user/organization data and causes interruptions. In the context of this study, ransomware activities are simulated by running certain scripts aimed at rapid file modifications and extension changes.
- **Rapid File Modification** It is an activity of performing changes on many files simultaneously. In this study, it is defined as the condition when more than 10 file modifications are carried out within a span of 5 seconds.
- **Resource Constrained Environments** The term implies computing environment with constraints in CPU capacity, memory, budget. In this research, it is operationalized as personal computer or local institution that does not have the capacity to run machine learning models or afford third-party software.

1.10 Organization of the Thesis

This thesis is divided into five chapters.Chapter 1 gives an introduction to the research through the presentation of background information, problem statement, purpose, research questions, objectives, significance, scope, limitations, and definitions of terms. Chapter 2 gives a review of the literature on ransomware threats, detection mechanisms, and endpoint security, especially those that relate to behavior-based security system models and empirical work. Chapter 3 focuses on the research methodology that includes experimental design, setting up of the Windows PowerShell environment, data collection methods for simulating the ransomware attack, and the evaluation measures. Chapter 4 analyzes the performance of the simulation and presents the answers to the research questions. Finally, chapter 5 interprets the results in view of the literature and makes conclusions about future research.

Chapter 2: Literature Review

2.1 Introduction

This chapter provides a systematic review of existing literature about ransomware, detection mechanisms, and automated responses. The main purpose of this review is the provision of the theoretical background for the current research and positioning of the proposed novel lightweight detection mechanism based on PowerShell in the modern cybersecurity research area.

As ransomware becomes more and more advanced in its methods of encrypting sensitive data belonging to organizations and individuals, it becomes difficult to combat such attacks using common methods of protection. The progression of ransomware attacks and the methods of their detection through the literature review will be discussed, with focus on common approaches to detection including the most advanced machine learning (ML) and deep learning (DL) solutions. Besides, the increasing importance of the endpoint protection and the usage of operating system native tools will be analyzed for the successful containment of the attack without reliance on additional third-party software.

In order to provide a systematic flow of information, the chapter is divided into sections, which cover the topic from general to specific issues. First, ransomware operation and its effect on the availability of the system will be discussed. Second, different kinds of detection mechanisms will be evaluated regarding their advantages and disadvantages, with a focus on resource consumption. Third, the role of the native operating system tools as a protective measure against ransomware attacks will be considered, focusing on Windows PowerShell. Fourth, a comparative analysis of similar recent literature (2021-2025) will be performed in order to identify the specific research gap.

2.2 Overview of Ransomware Threats

Among numerous types of malware, ransomware forms one of the significant and dangerous categories. It is a software, which targets the availability and confidentiality of resources on a digital device. Through the use of complex cryptography methods, ransomware makes it impossible to access OS or encrypt data on the computer. In order to decrypt it, users should pay the ransom usually in the form of cryptocurrencies.

In order to successfully deal with ransomware attacks, it is important to understand its kill chain cycle. First, the attacker uses various vectors such as malicious emails or attachments to penetrate into the target computer. Then, the software operates in a hidden manner and becomes persistent in the system while establishing connection with remote C2 servers in many cases. Afterward, the scan of local drive is conducted to find specific types of files for encryption. Usually, ransomware changes

the file extension, making it impossible to open files using any program. Finally, the extortion phase starts with the display of a ransom note to the user.

Recent surveys Fouzy et al [1] and Alzahrani et al [2] show that modern ransomware became more sophisticated, targeting specific companies or even individuals. Nowadays, threat actors utilize double extortion schemes when they first exfiltrate sensitive data from the machine and then encrypt it, threatening victims with a public exposure of their files if the ransom remains unpaid. Besides, nowadays threat actors use the technique of "Living off the Land" (LotL), which allows them to execute the fileless attacks through the utilization of legitimate administrative tools, such as Windows PowerShell and bypass the signature-based security perimeters without dropping any executables to disk Varlioglu et al [3].

This approach creates many challenges for businesses since the penetration by ransomware can result in system downtimes and reputational damage, and can lead to a loss of irreplaceable data. Although large corporations can afford multi-layer security architecture, small businesses and organizations suffer a lot due to the limited resources. In fact, the speed of file encryption does not provide any chances for humans to interfere in the process manually. Therefore, it is crucial to develop an effective solution that will be lightweight enough and provide immediate response to an incident at the endpoint level.

2.3 Ransomware Detection Techniques

Significant progress has been made in the research aimed at detecting and mitigating ransomware, mostly due to the fast-changing nature of the attack vectors. Accordingly, literature on this topic usually distinguishes between signature based, AI driven, and behavior based detection techniques.

2.3.1 Signature-Based Detection

Classic means of security are based on static analysis or predefined malware signatures. While these detection means are cheap from the computational perspective and efficient against the known threats, they are reactive by design. According to Alhuwayshil et al.[4] and Varlioglu et al. [3], signature-based techniques often fail to detect the newly developed zero-day threats. Besides, they are highly vulnerable to fileless ransomware and attack vectors exploiting the Windows utilities in order to bypass the perimeters Alhuwayshil et al., [4] Varlioglu et al., [3], Chew, [5]).

2.3.2 Machine Learning and AI Approaches

In order to overcome the shortcomings of static signatures, modern research tends to implement Machine Learning (ML) and Deep Learning (DL). Alzahrani et al. [2] and Fouzy et al. [1] review the state-of-the-art approaches, stressing the current problem of balancing the high detection rate with minimal overhead. For instance, Alhuwayshil et al. [4] proposed risk-aware classification models that used Windows API calls along with hybrid ML/DL solutions. Similar approaches were offered by Lee et al. [6]: they introduced a solution combining deep learning and behavioral features to detect unusual file modifications. Despite the high efficiency, these models imply considerable overhead, which makes them impractical for use in resource-restricted endpoints Alzahrani et al., [2].

2.3.3 Behavior-Based Detection

The behavior-based approach is widely discussed in literature due to its ability to detect malicious actions dynamically without requiring knowledge of the signature beforehand. Chew et al. [5] offered a real-time monitoring system based on system calls, while Xia and Zeng [7] proved that low-level system anomalies can be helpful when used in conjunction with behavior-based techniques to detect malicious activity. A relevant development in this sphere came from Harishni and Sowmyarani [8]: they proposed a monitoring tool capable of observing both file and process behavior to produce automatic actions with the help of entropy analysis. The techniques that monitor fast changes of the files and suspicious file extensions were proven to provide continuous protection with a low level of false positives [9, 10].

2.4 Endpoint Security and Native OS Tools

An increasing threat in today's environment is Living off the Land (LotL) attacks. The malware based on PowerShell and the fileless attacks represent particular difficulties because these threats operate in the RAM, evading the file-scanning procedures. Varlioglu et al. [3] studied the fileless cryptojacking attacks using PowerShell and underlined the need for continuous endpoint monitoring. The issues have been addressed in Ugarte et al. [11] and Arnold et al. [12], where the researchers focused on de-obfuscation and thorough analysis of the malicious PowerShell scripts. Thus, the dual role of native utilities, which can be used not only as attack vectors but also as monitoring tools, gives a chance to create defense systems natively.

2.5 Comparative Analysis of Related Studies

Table 1: Comparative Analysis of Recent Studies

Reference	**Proposed Methodology**	**Key Contributions**	**Limitations & Research Gap**
Alhuwayshil et al. (2025)	Hybrid ML/DL + Windows API Calls	High detection accuracy	High computational overhead; resource-intensive
Zhang et al. (2025)	Feature Selection + Hybrid Model	Effective feature optimization	Latency issues in real-time scenarios
Lee et al. (2023)	Deep Learning + Behavioral Features	Robust anomaly detection	Requires large datasets and significant GPU power
Harishni & Sowmyarani (2025)	Behavioral File/Process Monitoring	Integrated detection and automated response	Dependent on specific system configurations
Chew et al. (2024)	System Call-based Monitoring	Real-time threat detection	High complexity in analyzing low-level syscalls
Varlioglu et al. (2024)	PowerShell Attack Analysis	Deep insight into LotL attacks	Focuses on analysis, lacks active containment
Arnold et al. (2022)	Malware Rating System (PowerShell)	Effective script categorization	Static focus; lacks dynamic real-time response
Ugarte et al. (2021)	PowerShell Deobfuscation	Advanced script analysis	Does not address prevention or mitigation

2.6 Research Gap

Even though there have been significant achievements in ransomware detection techniques described in the literature, a careful examination shows a definite gap that should be addressed in this thesis. Modern approaches to ransomware detection, especially based on the Machine Learning and Deep Learning frameworks Alhuwayshil 2025; Zhang 2025; Lee 2023 [4, 10, 6], place a lot of emphasis on obtaining high accuracy rates through sophisticated feature engineering. Although successful in ideal conditions, these algorithms tend to be computationally expensive, dependent on GPU capacity, and require huge amounts of training data, thus making them not suitable for use on resource-limited devices like local organizational computers or regular office PCs.

Furthermore, a significant amount of research conducted nowadays tends to be purely analytical or is limited to ransomware detection and does not provide any mechanisms for containment or automatic mitigation of attacks. As shown in the comparative analysis (Table 1), research on ransomware that is based on PowerShell Varlioglu 2024; Ugarte 2021 [3, 11] gives a comprehensive understanding of the methods used by attackers but stops at the level of the analysis of de-obfuscation and fails to implement a proactive mechanism for containment of such attacks.

Also, many of the approaches suggested depend on external third-party applications, thus introducing additional management effort. There is a clear need to develop a "Living off the Land" approach that would utilize capabilities of the already pre-installed software, such as PowerShell of the Windows operating system, to detect malicious files and react to them instantly.

In this regard, there are identified three major gaps upon which the construction of the proposed model is based:

- **Resource efficiency:** The necessity to have a detection system that would use minimal CPU and memory resources.
- **Immediate response:** The lack of methods to integrate real-time detection with an automated response process, including process killing and alarms.
- **Lightweight infrastructure:** The necessity to have an efficient system that could operate properly using the available system infrastructure without resorting to advanced AI-based processing power.

In other words, the combination of these factors allows bridging the gap between a complicated detection model and the requirements of practical applications.

Chapter 3: Methodology

3.1 Introduction

This chapter presents the research methodology adopted to develop and evaluate a lightweight, behavior-based ransomware detection and automated response system using native Windows PowerShell. The methodology is designed to address the research problem, objectives, and questions established in Chapter One, while building on the research gap identified in Chapter Two.

The proposed system monitors file-system activities on a Windows 10 workstation and identifies potentially malicious behavior based on predefined behavioral indicators, including rapid file modifications and abnormal file-extension changes. When the configured detection conditions are satisfied, the PowerShell script initiates an automated response by attempting to terminate the suspicious process and recording the corresponding security event.

The study follows a quantitative experimental approach. A physical computer running Windows 10 will be used as the experimental environment, and a dedicated PowerShell simulation script will generate controlled ransomware-like behavior against a predefined collection of safe test files. The experiments will be conducted under different operating conditions to evaluate detection performance, response time, file protection, false-positive behavior, and system resource consumption.

The chapter describes the research design, experimental units, test data, measurement instruments, data collection procedures, implementation environment, experimental protocol, baseline comparisons, and analysis plan. It also discusses the ethical considerations and methodological limitations relevant to the study. The methodology is structured to support repeatable experiments and provide measurable evidence for evaluating the proposed system.

3.2 Nature of the Quantitative Research

This research is quantitative because the evaluation is based on numerical measurements obtained from repeated controlled experiments. The study does not use human participants or questionnaire-based instruments. Instead, the primary sources of evidence are PowerShell execution records, security logs, file-system states, process termination events, timestamps, and system resource measurements.

The proposed detection mechanism is rule-based rather than machine-learning-based. It applies predefined behavioral conditions to identify suspicious activity. In particular, the study examines rapid file modifications within a specified time window and unexpected file-extension changes. The existing implementation uses a threshold of more than ten file modifications within five seconds as one of its operational detection indicators.

The quantitative evaluation will focus on the following dimensions:

1. Detection performance: whether the system correctly identifies simulated ransomware-like behavior.
2. Response performance: the time required to detect suspicious activity and attempt to terminate the associated process.
3. File protection: the condition of the test files following each experimental run.
4. False-positive behavior: whether normal file-system activities incorrectly trigger the detection mechanism.
5. Resource consumption: the CPU and memory usage associated with running the detection script.

The numerical observations will be collected during the experiments and analyzed using descriptive statistics and appropriate comparisons between experimental conditions. Detection metrics such as precision, recall, and F1-score will be calculated only when the experimental records provide sufficient information to identify true positives, false positives, and false negatives.

3.3 Quantitative Research Design

The study adopts a controlled experimental design for evaluating a security detection and response system. The independent variable is the operating condition of the detection and response mechanism, while the dependent variables represent the observed security and performance outcomes.

The principal comparison will be conducted between simulated ransomware-like activity performed with the detection and response mechanism disabled and the same type of activity performed with the mechanism enabled. Additional benign activity tests will be performed to examine whether normal operations trigger inappropriate alerts or process termination.

The experiment will use a consistent collection of safe test files, predefined simulation settings, and a fixed detection configuration. Each scenario will be repeated to examine the consistency of the observed measurements. The order of the experimental runs and the condition of the test files will be controlled as far as practicable.

Table 2 summarizes the quantitative research design.

The purpose of this design is to determine whether the proposed system can detect the specified behavior, respond within a measurable period, reduce the impact of simulated ransomware-like activity on test files, and operate with acceptable resource overhead. The experiment is not intended to establish effectiveness against every ransomware family or against attacks outside the defined test scope.

Table 2: Quantitative Research Design

Design Element	**Description**
Research approach	Quantitative experimental evaluation of a lightweight ransomware detection and automated response system.
Research setting	A physical Windows 10 workstation operating in a controlled testing environment.
Proposed mechanism	A PowerShell-based, rule-driven system for monitoring suspicious file-system behavior and initiating an automated response.
Independent variable	Detection and response operating condition: disabled or enabled. Benign activity is also tested to evaluate false-positive behavior.
Dependent variables	Detection outcome, detection latency, response latency, file protection rate, false-positive count or rate, CPU usage, and memory usage.
Controlled conditions	Test-file collection, initial file state, simulation configuration, detection thresholds, operating system, hardware, and measurement procedures.
Data sources	PowerShell security logs, simulation logs, timestamps, process status, test-file verification records, and resource measurements.
Experimental repetition	Each scenario will be repeated several times. At least ten runs per scenario are proposed where practical, subject to pilot testing and available execution time.
Analysis approach	Descriptive statistics and comparisons between the experimental conditions. Classification metrics will be calculated where the recorded outcomes support them.

3.4 Study Population, Experimental Units, and Test Data

In this study, the experimental population consists of the predefined classes of file-system activity that the proposed system is intended to distinguish: simulated ransomware-like behavior and legitimate file-system activity. Because the research evaluates software behavior rather than human participants, the experimental units are individual test runs and their associated security events.

The test data will consist of a controlled collection of safe files used by the simulation script. The initial research plan specifies a collection of 100 test files. These files will be stored in a dedicated test directory and will not contain valuable personal or organizational data.

The simulation script will operate only on the designated test directory. Before each experimental run, the test files will be restored to their original state or recreated from clean copies. This procedure is necessary to ensure that each run begins under comparable conditions and that previous runs do not influence subsequent measurements.

The test collection and the simulated activities will be documented so that each run can be associated with its initial conditions, scenario type, detection setting, and resulting file state.

Table 3 identifies the units and sources of evidence used in the study.

The experimental unit must be distinguished from the individual file. For example, a single run may involve multiple file operations but still represent one experimental run. This distinction will be maintained when calculating trial-level detection metrics to avoid treating every file operation as an independent detection trial.

3.5 Measurement Instruments and Operational Definitions

The study uses software-based measurement instruments rather than human research instruments. The primary instrument is the developed PowerShell detection and response script. The simulation script provides controlled test activity, while the logging and verification procedures provide evidence for evaluating the system's behavior.

The detection script records relevant events according to its implemented logging capabilities. The simulation records or establishes the expected scenario and its execution period. File-state verification is used to assess the condition of the test files before and after each run. System resource measurements are collected using an appropriate Windows-compatible monitoring method, with the measurement method and sampling interval documented before the main experiments.

To ensure consistency, the study uses operational definitions for the principal vari-

Table 3: Experimental Units and Data Sources

Element	**Operational Definition**	**Evidence Collected**
Experimental unit	One complete execution of a defined test scenario.	Run identifier, scenario type, detection setting, start time, and end time.
Test files	The predefined collection of 100 safe files used in the experiments.	Initial and final file counts, file states, and verification results.
Simulated malicious activity	File operations generated by the dedicated ransomware-behavior simulation script.	Simulation timestamps, operation records, and the expected scenario classification.
Benign activity	Legitimate file operations performed to examine false-positive behavior.	Activity type, execution timestamps, alerts, and any process termination events.
Detection event	An event in which the PowerShell detection mechanism identifies behavior that satisfies its configured conditions.	Detection timestamp, triggered rule or indicator, and associated process information where available.
Response event	An automated response initiated after detection.	Response initiation time, process termination attempt, termination outcome, and notification records.
Resource observation	A measurement of system resource consumption during a defined observation period.	CPU and memory measurements, observation interval, and experimental condition.

ables. The threshold of more than ten file modifications within five seconds is treated as the configured rapid-modification indicator described in the research scope. The extension-change indicator is evaluated according to the conditions implemented in the script. These indicators are not assumed to detect all ransomware behavior; their performance will be assessed through the controlled experiments.

Table 4 defines the main variables and their measurement methods.

The file protection rate is calculated as follows:

$$FileProtectionRate(\%) = \frac{NumberofTestFilesVerifiedasIntact}{TotalNumberofTestFiles} \times 100$$

A file will be classified as intact according to the verification criteria established before the main experiments. These criteria will consider the file's expected state and whether its content or name has been changed by the simulation. The definition will be applied consistently to all experimental conditions.

Detection latency and response latency will be recorded separately. Detection latency measures the time required to identify the suspicious behavior, whereas response latency measures the time between detection and confirmed process termination. If a termination attempt fails or cannot be confirmed, the outcome will be recorded as a failure rather than assigning an artificial termination time.

The resource measurements will be collected using the same monitoring method and sampling interval for the baseline and enabled conditions. The monitoring interval and any measurement overhead will be documented to support fair comparisons.

3.6 Data Collection Procedures

Data collection will follow a predefined sequence to maintain consistency across experimental runs. Each run will receive a unique identifier that connects the scenario configuration, simulation activity, detection logs, response events, file verification results, and resource observations.

Before each run, the researcher will verify the initial condition of the test directory, confirm the intended detection setting, and record the scenario configuration. The simulation will then be executed according to the selected test case. After execution, the researcher will collect the generated logs, inspect the process status, verify the test files, and record the system resource measurements.

The test files will be restored to their initial state before another run begins. If a run is affected by an execution error, incomplete logging, or an unexpected system interruption, the reason will be documented. Such a run will not be silently discarded; its status and treatment in the analysis will be recorded.

Table 4: Operational Definitions and Measurement Methods

Variable	**Operational Definition**	**Unit**	**Measurement Method**
Detection outcome	Whether the system identifies a predefined simulated ransomware-like trial.	Binary: detected/not detected	Comparison of expected scenario labels with detection logs.
Detection latency	Elapsed time between the first relevant simulated malicious operation and the first corresponding detection event.	Seconds	Difference between the relevant event timestamps.
Response latency	Elapsed time between the detection event and the confirmed process termination, where termination succeeds.	Seconds	Difference between detection and process-termination timestamps.
Termination outcome	Whether the suspicious process is confirmed to have terminated following the response attempt.	Binary: success/failure	Process status and response logs.
File protection rate	Percentage of the designated test files verified as intact after a run.	Percentage	Number of intact test files divided by the total number of test files, multiplied by 100.
False-positive rate	Proportion of benign trials incorrectly classified as malicious.	Percentage	False-positive benign trials divided by all benign trials, multiplied by 100.
CPU usage	CPU utilization observed during the defined experimental monitoring period.	Percentage	Consistent system-monitoring procedure applied across test conditions.
Memory usage	Memory consumed by the detection process or mechanism during the defined observation period.	MB or another consistently recorded unit	Consistent Windows-compatible process-monitoring procedure.

Table 5 presents the data collection sequence.

Table 5: Data Collection Procedure

Step	**Procedure**	**Recorded Evidence**
1	Prepare the test environment.	Operating system and PowerShell versions, hardware details, test directory, and script versions.
2	Restore and verify the test files.	Run identifier, initial file count, initial file state, and verification status.
3	Configure the experimental condition.	Scenario type, detection and response setting, and configuration parameters.
4	Execute the selected scenario.	Simulation start time, relevant file operations, and scenario completion or interruption.
5	Collect detection and response evidence.	Detection time, triggered indicator, response initiation, process termination outcome, and alerts.
6	Measure the final file state and resource usage.	Number of intact files, file changes, CPU observations, and memory observations.
7	Validate and store the run record.	Completeness checks, associated log files, anomalies, and run status.
8	Restore the test environment.	Confirmation that the test files are ready for the next independent run.

The collected records will be organized in a structured format, such as CSV files, to facilitate subsequent analysis. The existing security log file, `Security_Logs.csv`, will be used if it is the actual output file produced by the implementation. The final file names and recorded fields will be documented according to the implemented scripts.

Each experimental record should include, where available, the run identifier, scenario, detection condition, relevant timestamps, detection outcome, response outcome, file verification results, resource measurements, and any execution anomalies. Timestamps will use a consistent format to allow elapsed-time calculations.

3.7 Experimental Environment and Implementation Details

The experiments will be conducted on a physical Dell XPS 15 computer running Microsoft Windows 10. The workstation has 32 GB of RAM and 1 TB of storage. Using a physical computer allows the proposed system to be evaluated under an actual Windows operating environment rather than relying exclusively on a simulated hardware environment.

The detection and response mechanism is implemented using Windows PowerShell. Its purpose is to monitor the specified behavioral indicators, identify suspicious ac-

tivity according to the configured rules, and attempt to terminate the associated suspicious process. The system also records security events and generates the notifications supported by the implementation.

A separate PowerShell simulation script will be used to generate controlled ransomware-like behavior against the safe test files. The simulation is intended to reproduce selected observable behaviors rather than execute real ransomware or encrypt valuable user data.

The exact Windows build, PowerShell version, processor model, script versions, detection thresholds, monitoring method, and other relevant configuration details will be recorded during the experimental setup. These details are necessary for reproducibility and will be reported in the final experimental documentation.

Table 6 summarizes the known experimental environment and the additional details to be recorded.

Table 6: Experimental Environment and Implementation Configuration

Component	**Configuration**
Physical device	Dell XPS 15 laptop.
Operating system	Microsoft Windows 10; exact edition, build, and update level to be recorded.
Memory	32 GB RAM.
Storage	1 TB storage capacity.
Processor	Exact processor model and relevant specifications to be recorded during setup.
Detection and response technology	Windows PowerShell using the developed behavior-based detection and automated response script.
Simulation technology	A dedicated PowerShell script that generates controlled ransomware-like behavior against safe test files.
Test data	A predefined collection of 100 safe test files in a dedicated test directory.
Detection indicators	Rapid file modifications and abnormal file-extension changes, according to the implemented detection rules.
Rapid-modification threshold	More than ten file modifications within a five-second window, as defined in the research scope.
Logging	PowerShell security-event records and associated simulation, response, and verification records.
Resource monitoring	A consistent Windows-compatible monitoring method; the tool or command, sampling interval, and measurement scope will be documented.

Before the main experiments, the researcher will verify that the scripts execute as intended, that the test directory is correctly isolated from valuable data, and that the relevant events are recorded. The configuration will be kept consistent across comparable experimental conditions. Any necessary change to a script or

its detection rules will be documented and, where it affects the behavior being measured, the affected experiments will be repeated under the revised configuration.

3.8 Experimental Design and Protocol

The experimental protocol is designed to evaluate detection, response, file protection, false-positive behavior, and resource consumption under controlled conditions. The study will use three principal test scenarios.

1. **Benign activity with detection enabled:** Legitimate file-system operations will be performed to determine whether the detection mechanism generates inappropriate alerts or terminates a legitimate process.
2. **Simulated ransomware-like activity with detection disabled:** The simulation script will operate on the safe test files without the proposed detection and response mechanism intervening. This condition provides a baseline for observing the file changes caused by the simulation.
3. **Simulated ransomware-like activity with detection and response enabled:** The same defined simulation behavior will be executed while the proposed mechanism monitors activity and attempts to respond when its detection conditions are satisfied.

The detection-disabled condition is intended to establish a baseline for the impact of the simulation. The detection-enabled condition will be compared with this baseline to assess whether detection and process termination are associated with improved file protection. The benign scenario will provide evidence about false positives and unintended interference with legitimate activity.

The simulation parameters, test-file collection, and initial file state will be kept consistent across comparable runs. Each scenario will be repeated several times. At least ten runs per scenario are proposed where practical, with the final number documented before the main data collection begins. A preliminary pilot run will be used to identify problems in logging, timestamp collection, file restoration, and process-state verification before the main experiment is conducted.

Table 7 defines the experimental scenarios.

For each scenario, the experimental run will follow a common procedure:

1. Assign a unique run identifier and record the scenario configuration.
2. Restore the test files to their predefined initial state and verify their integrity.
3. Record the detection and response settings before execution.

Table 7: Experimental Scenarios

Scenario	**Activity**	**Purpose**	**Main Measurements**
S1: Benign	Legitimate file-system operations with detection enabled.	Evaluate false positives and unintended process termination.	Alerts, false-positive trials, process actions, CPU, and memory.
S2: Baseline	Simulated ransomware-like behavior with detection and response disabled.	Establish the effect of the simulation without intervention from the proposed system.	File changes, final file state, and simulation duration.
S3: Protected	The same defined simulated behavior with detection and response enabled.	Evaluate detection, automated response, and file protection.	Detection latency, response latency, termination outcome, file protection, CPU, and memory.

4. Start the resource-monitoring procedure and record the relevant start time.
5. Execute the selected benign or simulated activity.
6. Collect the detection and response events generated during the run.
7. Record the process termination outcome, where applicable.
8. Verify the final state of the test files.
9. Stop resource monitoring and record the final measurements.
10. Validate the completeness of the logs and document any anomalies.
11. Restore the test directory before the next run.

The experimental procedure will be applied consistently across all scenarios. The researcher will not alter the detection thresholds selectively for individual runs after observing their outcomes. If a configuration change is necessary, it will be documented as a new experimental version and evaluated consistently.

The primary experimental comparison will be between S2 and S3. The S1 results will be used to evaluate false-positive behavior and potential interference with normal activity. These comparisons will be based on recorded measurements rather than assumptions about the expected effectiveness of the proposed system.

3.9 Baseline Selection and Comparison Strategy

The main baseline is the execution of the same controlled simulation while the proposed detection and response mechanism is disabled. This baseline is appropriate because the research objective is to determine how the implemented mechanism changes the observable outcomes of simulated ransomware-like activity.

The enabled and disabled conditions will use the same test-file collection and comparable simulation settings. The initial state of the test files will be verified before every run. Where feasible, the execution order will be varied to reduce the influence of systematic changes in background system activity.

The study does not claim to provide a direct benchmark against commercial endpoint detection and response products or a comprehensive comparison with state-of-the-art ransomware detection systems. The central evaluation concerns the performance of the proposed PowerShell-based mechanism relative to the defined experimental baseline.

Table 8 presents the baseline comparison strategy.

Table 8: Baseline and Experimental Comparison

Comparison Element	**Baseline Condition**	**Detection-Enabled Condition**
Detection and response	Disabled for the experimental run.	Enabled according to the implemented rules.
Simulation	The defined safe ransomware-behavior simulation.	The same defined simulation under comparable conditions.
Test files	The same predefined test-file collection.	The same predefined test-file collection restored to its initial state.
File outcome	Changes resulting from the simulation without intervention from the proposed mechanism.	Changes remaining after detection and any successful response.
Timing	Simulation duration and relevant file-operation timestamps.	Detection latency, response latency, and simulation duration.
System overhead	Resource measurements collected using the same procedure.	Resource measurements collected using the same procedure.

The comparison will examine differences in file protection, execution time, and resource consumption. The results will be interpreted within the limits of the defined simulation. Because the experiment uses simulated behavior and a specific rule-based implementation, the baseline comparison cannot establish effectiveness against all real-world ransomware variants.

3.10 Statistical and Experimental Analysis Plan

The collected data will be analyzed using descriptive statistics and comparisons between experimental conditions. The analysis will address the research questions by examining detection outcomes, response timing, file protection, false positives, and resource consumption.

For continuous variables, including detection latency, response latency, CPU usage, and memory usage, the analysis will report the number of valid observations, mean, median, and standard deviation where appropriate. Minimum and maximum values may also be reported to illustrate the observed range. Where a run does not produce a valid timing measurement, the reason will be documented and the observation will not be treated as a successful response.

For categorical outcomes, including detection success, process termination success, and file-integrity status, the analysis will report counts and percentages. The results will distinguish between detection events, response attempts, confirmed termination outcomes, and final file states.

Detection performance will be evaluated using trial-level classification outcomes. Each predefined trial will have an expected label based on its scenario and a recorded system outcome. A trial will be classified as a true positive (TP) when simulated malicious behavior is correctly detected, a false positive (FP) when benign activity is incorrectly classified as malicious, a false negative (FN) when simulated malicious behavior is not detected, and a true negative (TN) when benign activity is correctly treated as benign.

The following metrics will be calculated when the recorded trial outcomes support their calculation:

$$Precision = \frac{TP}{TP + FP} \times 100$$

$$Recall = \frac{TP}{TP + FN} \times 100$$

$$F1 - score = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

$$FalsePositiveRate = \frac{FP}{FP + TN} \times 100$$

If a denominator is zero, the corresponding metric will be reported as undefined rather than assigned an arbitrary value. These metrics will be used to describe the

performance of the rule-based detector; they do not imply that a machine-learning classifier is part of the system.

The accuracy metric may also be reported if the test design provides valid true-positive, true-negative, false-positive, and false-negative counts:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \times 100$$

The file protection rate will be calculated separately for each run using the formula defined in Section 3.4. The study will compare the file protection measurements from the baseline and detection-enabled conditions. The comparison will also report the number of successful and unsuccessful process termination outcomes.

Detection latency will be measured from the first relevant simulated malicious operation to the first corresponding detection event. Response latency will be measured from the detection event to confirmed process termination. These two measurements will not be combined because they represent different stages of the response process.

Resource consumption will be evaluated using CPU and memory observations collected under comparable conditions. The analysis will report the measurement scope and sampling interval, and will compare the observed resource usage with the detection mechanism disabled and enabled. Any system background activity or measurement limitations that may affect the observations will be documented.

Table 9 links the research questions to the planned measurements and analysis.

The primary analysis will be descriptive. Inferential statistical tests will only be added if the final number of independent observations, distribution of the measurements, and experimental design justify their use. The analysis will not assume statistical significance without conducting an appropriate test and documenting its assumptions.

The results and numerical findings will be presented in Chapter Four. This chapter defines the measurements and analysis procedures without presuming that the proposed system will achieve a particular detection rate, response time, or resource-consumption level.

3.11 Ethical and Safety Considerations

The experiments will use a controlled simulation script and a dedicated collection of safe test files. The study will not intentionally execute real ransomware or target external systems, third-party devices, or valuable personal files.

The simulation will be restricted to the designated test directory. Before each run, the researcher will verify that the test data can be restored and that the simulation is

Table 9: Research Questions and Analysis Plan

Research Question	**Main Variables**	**Data Source**	**Analysis Method**
RQ1: How accurately does the system identify ransomware behavior?	Detection outcome, TP, FP, FN, TN, precision, recall, and F1-score where calculable.	Simulation records and detection logs.	Counts, percentages, and classification metrics.
RQ2: How quickly does the automated response contain simulated activity and protect files?	Detection latency, response latency, termination outcome, and file protection rate.	Detection logs, process status, timestamps, and file verification records.	Mean, median, standard deviation where appropriate, counts, percentages, and baseline comparison.
RQ3: What is the impact on system resources and false-positive behavior during normal activities?	CPU usage, memory usage, false-positive trials, and unintended process termination.	Resource-monitoring records, benign scenario logs, and process status.	Descriptive statistics, counts, percentages, and comparisons across conditions.

configured to operate only on the intended files. Original personal and organizational data will not be used as experimental targets.

Because the proposed system can attempt to terminate a process, the experiments will be performed only under controlled conditions. The researcher will verify the process associated with the test scenario and will document the outcome of each termination attempt. A termination request will not automatically be treated as proof that the process was successfully stopped.

The logs and collected measurements will be used for research and system evaluation. Any logs containing sensitive system information will be stored securely and will not be shared unnecessarily. The experimental records will be retained in a structured format to support verification of the reported findings.

These precautions are intended to reduce the risk of unintended data loss, process interruption, and exposure of sensitive information during the experiments.

3.12 Limitations and Threats to Validity

Several limitations may affect the interpretation and generalizability of the experimental results.

Internal validity: Background processes, system updates, resource fluctuations, and differences in execution timing may influence the measured results. To reduce

these effects, the experiments will use consistent configurations, repeated runs, a predefined test-file collection, and documented measurement procedures.

Construct validity: The simulation reproduces selected ransomware-like behaviors rather than the complete behavior of real ransomware. Consequently, detection performance against the simulation may not represent performance against every ransomware family or every attack technique. The behavioral indicators and the definitions of successful detection and file protection will be specified before the main experiments.

External validity: The evaluation will be conducted on a single physical Dell XPS 15 computer running Windows 10. The results may differ on other hardware configurations, Windows versions, workloads, and system environments. The findings should therefore be interpreted as an evaluation of the proposed mechanism under the specified experimental conditions.

Measurement validity: Timing accuracy depends on the quality and consistency of event timestamps. Resource measurements may also be affected by the monitoring method and its sampling interval. The study will document the methods used to collect these measurements and will distinguish between attempted and confirmed process termination.

Implementation validity: The behavior of the system depends on the implemented PowerShell rules and response logic. Changes to the script, its configuration, or the simulation may influence the results. Script versions and relevant configuration parameters will therefore be recorded, and changes affecting the measured behavior will be documented.

Statistical limitations: Repeated runs can improve the reliability of descriptive comparisons, but a limited number of scenarios or runs may not represent the full range of benign and malicious behavior. The number of completed runs, excluded or invalid runs, and reasons for exclusion will be reported transparently.

These limitations do not prevent the system from being evaluated experimentally. Rather, they define the scope within which the results can be interpreted and identify areas that may require further investigation.

3.13 Chapter Summary

This chapter presented the quantitative experimental methodology for evaluating the proposed lightweight, behavior-based ransomware detection and automated response system implemented using native Windows PowerShell.

The study uses a physical Windows 10 workstation, a controlled simulation script, and a predefined collection of safe test files. Three principal scenarios will be evaluated: benign activity with detection enabled, simulated ransomware-like activity

with detection disabled, and simulated ransomware-like activity with detection and response enabled.

The methodology defines the experimental units, operational variables, data collection procedures, baseline comparison, and analysis plan. The principal measurements include detection performance, detection and response latency, process termination outcomes, file protection rate, false-positive behavior, CPU usage, and memory usage.

The collected data will be analyzed using descriptive statistics and appropriate comparisons between experimental conditions. The results will be reported in Chapter Four, where the observed performance of the proposed system will be assessed against the research questions and objectives established in Chapter One. "'

References

[1] N. Fouzy, M. Alyammahi, A. Alyammahi, M. Elhadef, and M. Hadded, "Ransomware detection techniques: A survey of evolution, trends, and open challenges," in *Proc. 14th IFIP/IEEE Int. Conf. on Performance Evaluation and Modeling in Wired and Wireless Networks (PEMWN)*, Paris, France, 2025, pp. 1–10.

[2] S. Alzahrani, Y. Xiao, S. Asiri, J. Zheng, and T. Li, "A survey of ransomware detection methods," *IEEE Access*, vol. 13, pp. 57 943–57 982, 2025.

[3] R. Varlioglu, N. Elsayed, E. R. Varlioglu, and M. Ozer, "The pulse of fileless cryptojacking attacks: Malicious powershell scripts," in *IEEE SoutheastCon*, 2024.

[4] S. Alhuwayshil, S. Ramachandran, and K. Kim, "Enhancing ransomware threat detection: Risk-aware classification via windows api call analysis and hybrid ml/dl models," *Journal of Cybersecurity and Privacy*, vol. 5, no. 4, p. Art. 96, Nov 2025.

[5] C. J. W. Chew, V. Kumar, P. Patros, and R. Malik, "Real-time system call-based ransomware detection," *International Journal of Information Security*, vol. 23, pp. 1839–1858, 2024.

[6] J. Lee, H. Kim, J. Park, and S. Lee, "Ransomware detection using deep learning and behavioral features," *IEEE Transactions on Information Forensics and Security*, vol. 18, pp. 2345–2358, 2023.

[7] Y. Xia and Q. Zeng, "Real-time system call-based ransomware detection," *International Journal of Information Security*, vol. 18, pp. 267–281, 2019.

[8] L. Harishni and C. N. Sowmyarani, "Real-time ransomware detection and response system using behavioural file and process monitoring," in *Proceedings*

of the 2025 9th International Conference on Computational System and Information Technology for Sustainable Solutions (CSITSS), 2025, pp. 1–7.

[9] D. S. Kumar, R. S. Rao, and M. T. Khan, "Exploring ransomware detection based on artificial intelligence and machine learning," *Procedia Computer Science*, vol. 252, pp. 548–556, 2025.

[10] K. Zhang, Y. Wang, U. A. Bhatti, Y. Zhou, and M. Jin, "Enhanced ransomware attacks detection using feature selection and hybrid model," *Journal of Big Data*, vol. 12, p. Art. 245, 2025.

[11] A. Ugarte, D. Maiorca, F. Cara, and G. Giacinto, "Powerdrive: Accurate deobfuscation and analysis of powershell malware," *Springer International Journal on Secure Software Integration and Reliability*, pp. 112–127, 2021.

[12] D. Arnold, C. David, and J. Saniie, "Powershell malware analysis using a novel malware rating system," in *2022 IEEE International Conference on Cybersecurity and Resilience*, 2022, pp. 45–54.